

Entrega 1: Panel con Conexión de Wallet

(10 pts)

Objetivo: Construir una pequeña aplicación React que se conecte a una wallet Ethereum real y lea datos en vivo desde la Blockchain (testnet).

En esta entrega no se requiere escribir ningún contrato inteligente. El foco está en el stack de **frontend** que se utilizará a lo largo del curso.

Qué Construir

Una página única con dos secciones:

1. Panel de Cuenta

Mostrar lo siguiente para la billetera conectada:

- Nombre ENS (si está disponible) o dirección abreviada.
- Saldo de ETH nativo, formateado a 4 decimales.
- El número de bloque actual, actualizado en tiempo real (o mediante un intervalo corto de actualización).

2. Saldos de Tokens

Elegir dos tokens ERC-20 cualquiera desplegados en Sepolia (por ejemplo, USDC o LINK de faucet). Para cada uno:

- Leer el nombre y símbolo del contrato (`name()`, `symbol()`).
- Leer el saldo de la billetera conectada (`balanceOf(address)`).
- Mostrar ambos valores correctamente formateados (utilizar `decimals()` del token).

El usuario debe poder desconectar y reconectar su billetera. Cuando no haya billetera conectada, se debe mostrar únicamente un botón de conexión destacado y ningún otro contenido.

Stack Requerido

Librería	Propósito
React o equivalente	Framework de UI
wagmi v2	Conexión de billetera, lectura de contratos
viem o ethers	Primitivas de Ethereum, codificación de ABI
RainbowKit o ConnectKit	UI de conexión de billetera
TypeScript	Obligatorio. El uso de any resta puntos

Criterios de Evaluación

- La billetera se conecta y desconecta correctamente mediante la UI de conexión.
 - El saldo de ETH y el número de bloque son datos en vivo (no valores estáticos).
 - El nombre y el saldo de ambos tokens ERC-20 se leen desde Sepolia.
 - No hay errores de TypeScript (`tsc --noEmit` debe ejecutarse sin errores).
 - Los componentes están organizados de forma coherente — ningún archivo debe superar las ~200 líneas.
-

Entrega

Un repositorio de GitHub con un archivo [README.md](#) que incluya:

- Instrucciones para ejecutar el proyecto localmente.
 - Las direcciones de los dos tokens en Sepolia que se utilizaron.
-

Recursos

- Faucet de Sepolia - [Opción 1](#), [Opción 2](#)